

Intermediate Code Generation + Basic Block Formation + Control Flow Graph for If-Else and While : A Case Study

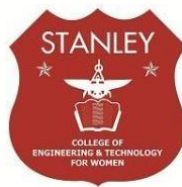
*Case study Report submitted in partial fulfilment of the requirements for the award of
the Degree of B.E in Computer Science and Engineering*

By

Jessica	160623733146
K.Vignitha	160623733150
Shaik Karishma	160623733177
T. Akshitha	160623733185
V. Lasya	160623733187
Pedaprolu S S L Katyayani	160624733315

Under the Guidance of

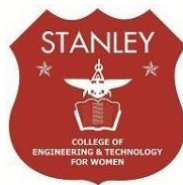
Assistant Professor, Department of Computer Science & Engineering



**Department of Computer Science and Engineering
Stanley College of Engineering & Technology for
Women(Autonomous)**

Chapel Road, Abids, Hyderabad – 500001

(Affiliated to Osmania University, Hyderabad, Approved by AICTE, Accredited by
NBA & NAAC with A Grade)



Stanley College of Engineering & Technology for Women

(Autonomous)

Chapel Road, Abids, Hyderabad – 500001

**(Affiliated to Osmania University, Hyderabad, Approved by AICTE, Accredited
by NBA & NAAC with A Grade)**

CERTIFICATE

This is to certify that the case study report entitled **“Intermediate Code Generation + Basic Block Formation + Control Flow Graph for If-Else and While”** is being submitted by

Jessica	160623733146
K.Vignitha	160623733150
Shaik Karishma	160623733177
T Akshitha	160623733185
V. Lasya	160623733187
Pedaprolu S S L Katyayani	160623733315

In partial fulfilment for the award of the Degree of Bachelor of Engineering in Computer Science & Engineering to the Osmania University, Hyderabad is a record of bonafide work carried out under my guidance and supervision. The results embodied in this Problem statement report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Guide

Mrs. Gnana Prasuna
Professor, Dept. of CSE

Head of the Department

Dr. R Madana Mohana
Professor & Head , Dept. of CSE



STANLEY COLLEGE OF ENGINEERING AND TECHNOLOGY FOR WOMEN
(Autonomous)
Chapel Road, Abids, Hyderabad.
Department of Computer Science & Engineering

Institute Vision:

Empowering girl students through professional integrated with values and character to make an Impact in the world

Institute Mission:

M1: *Enabling quality engineering education for girl students to make them competent and confident to succeed in professional practice and advanced learning.*

M2: *Providing state-of-art-facilities and resources towards world class education.*

M3: *Integrating qualities like humanity, social values, ethics, leadership towards their contribution to society.*

Department Vision:

Empowering girl students with the contemporary knowledge in Computer Science and Engineering for their success in life.

Department Mission:

M1: *To impart quality education for girl students to learn and practice various hardware and software platforms prevalent in industry.*

M2: *To achieve self-sustainability and overall development through Research and Development activities.*

M3: *To provide education for life by focusing on the inculcation of human & moral values through and honest and scientific approach*

M4: *To groom students with good attitude, team work and personality skills.*

Programme Educational Objectives

PEO1: Our graduates shall have enhanced skills and contemporary knowledge in software and hardware technologies for professional excellence, towards successful employment, advanced learning and research.

PEO2: Our graduates shall have life-long learning attitude, innovation and creativity to master latest technologies, devise solutions for realistic and social issues in the society.

PEO3: Our graduates have good attitude and personality skills, ethical values, teamwork and leadership skill towards professionalism and ethical practices within the organization and the society.

Program Outcomes:

P01: Engineering Knowledge: Apply knowledge of mathematics and science, with fundamentals of Computer Science & Engineering to be able to solve complex engineering problems related to CSE.

P02: Problem Analysis: Identify, Formulate, review research literature and analyze complex engineering problems related to CSE and reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.

P03: Design/Development of solutions: Design solutions for complex engineering problems related to CSE and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural societal and environmental considerations.

P04: Conduct Investigations of Complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

P05: Modern Tool Usage: Create, Select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to computer science related complex engineering activities with an understanding of the limitations.

P06: The Engineer and Society: Apply Reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the CSE professional engineering practice.

P07: Environment and Sustainability: Understand the impact of the CSE professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.

P08: Ethics: Apply Ethical Principles and commit to professional ethics and responsibilities and norms of the engineering practice.

P09: Individual and Team Work: Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary Settings.

P010: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large such as able to comprehend and with write effective reports and design documentation, make effective presentations and give and receive clear instructions.

P011: Problem statement Management and Finance: Demonstrate knowledge and understanding of the engineering management principles and apply these to one's own work, as a member and leader in a team, to manage Problem statements and in multi disciplinary environments.

P012: Life-Long Learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning the broadest context of technological change.

Program Specific outcomes:

PS01: Problem-Solving Skills: The ability to apply standard practices and strategies in software Problem statement development using open-ended programming environments to deliver a quality product for the benefit of students.

PS02: Design, Implement, Test and Evaluate a computer system, component or algorithm to meet desired needs and to solve a computational problem.

ABSTRACT

The study explains how high-level programming constructs are transformed into intermediate code using three-address code representation. It also discusses the identification and formation of basic blocks, which are sequences of instructions having a single entry and single exit point. These basic blocks help in improving program analysis and optimization techniques.

Intermediate Code Generation is an important phase in compiler design that converts source code into an intermediate representation that is easier for optimization and target code generation. This case study focuses on the concepts of Intermediate Code Generation, Basic Block Formation, and Control Flow Graph (CFG) construction for control statements such as If-Else and While loops.

Further, the report describes the construction of Control Flow Graphs, where nodes represent basic blocks and edges represent the flow of control between them. Special emphasis is given to If-Else conditional statements and While iterative statements to understand branching and looping behavior in programs. The CFG helps in analyzing execution paths, detecting unreachable code, and supporting compiler optimizations.

Tables of Contents

1. Introduction

1.1 About Problem statement

1.2 Objectives of the Problem statement

1.3 Scope of the Problem statement

1.4 Advantages

1.5 Disadvantages

1.6 Applications

1.7 Hardware and Software Requirements

2. Implementation

2.1 Code Snippets Implementation

3. Result Analysis

4. References

LIST OF FIGURES

Figure 2.1 Output 1

Figure 2.2 Output 2

1. INTRODUCTION

Compiler Design is an important area in computer science that focuses on translating high-level programming languages into machine-level instructions that can be executed by a computer. One of the major phases in a compiler is Intermediate Code Generation, where the source program is converted into an intermediate representation that is independent of machine architecture. This representation makes the process of optimization and final code generation easier and more efficient.

In compiler optimization, understanding the flow of a program is very important. For this purpose, compilers divide the intermediate code into Basic Blocks, which are groups of consecutive instructions having a single entry point and a single exit point. Basic block formation helps in simplifying program analysis and improving optimization techniques.

Another essential concept is the Control Flow Graph (CFG), which represents the flow of control between different basic blocks in a program. In a CFG, each node represents a basic block and edges represent the transfer of control from one block to another. This graphical representation helps in analyzing branching, looping, and execution paths in programs.

The study explains how these statements are represented in intermediate code, divided into basic blocks, and visualized using control flow graphs. The objective of this report is to provide a clear understanding of how compilers internally process conditional and looping statements, and how these techniques contribute to efficient program execution and optimization.

1.1 About Problem statement:

This case study focuses on generating **Intermediate Code (Three Address Code - TAC)** from high-level constructs like if-else and while statements. It also includes the formation of **Basic Blocks** and construction of a **Control Flow Graph (CFG)**.

Intermediate code acts as a bridge between source code and machine code, enabling optimization and efficient execution. Basic blocks and CFG help in analyzing the flow of control in a program.

1.2 Objectives of the Problem statement

- To understand intermediate code generation
- To generate **Three Address Code (TAC)**
- To identify and construct **Basic Blocks**
- To build **Control Flow Graph (CFG)**
- To analyze program flow for optimization
- To implement these concepts using Java GUI

1.3 Scope of the Problem statement

- Supports simple statements like assignment, if-else, and while
- Generates simplified TAC
- Constructs basic blocks manually or logically
- Displays a simple CFG structure
- Does not include advanced optimizations or real compiler backend

1.4 Advantages

- Helps in understanding compiler middle phase
- Improves knowledge of program flow analysis
- Useful for optimization techniques
- Visual representation using GUI improves clarity
- Forms foundation for advanced compiler design

1.5 Disadvantages

- Simplified implementation (not fully automated)
- Limited to basic constructs
- CFG visualization is textual, not graphical
- No advanced optimizations like loop unrolling or register allocation

1.6 Applications

- Compiler construction
- Code optimization tools
- Static code analysis tools
- Program verification systems
- IDE internal processing

1.7 Hardware & Software Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4GB minimum
- Storage: 500MB free space

Software Requirements

- Java JDK 8 or above
- IDE (Eclipse / IntelliJ / VS Code)
- Operating System: Windows/Linux/Mac

2. IMPLEMENTATION

2.1 Code Snippets Implementation

1. GUI Creation

```
JTextArea input = new JTextArea("a = b + c * d;");
```

```
JTextArea output = new JTextArea();
```

```
JButton generate = new JButton("Generate TAC");
```

2. Event Handling

```
generate.addActionListener(e -> {  
    String code = input.getText();  
    String tac = generateTAC(code);  
    output.setText(tac);  
});
```

3. Intermediate Code Generation (TAC)

```
public static String generateTAC(String code) {  
    return "t1 = c * d\n" +  
        "t2 = b + t1\n" +  
        "a = t2\n";  
}
```

4. Basic Block Formation

```
String basicBlock = "Basic Block:\nBlock1 -> t1, t2, a";
```

5. Control Flow Graph (CFG)

```
String cfg = "CFG:\n[Start] -> Block1 -> [End]";
```

6. Combined Output

```
return tac + "\n" + basicBlock + "\n\n" + cfg;
```

2.1 Code Implementation

JAVA

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

public class TAC_GUI {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Intermediate Code Generator");

        frame.setSize(500, 400);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTextArea input = new JTextArea("a = b + c * d;");

        JTextArea output = new JTextArea();

        JButton generate = new JButton("Generate TAC");

        generate.addActionListener(e -> {

            String code = input.getText();

            String tac = generateTAC(code);

            output.setText(tac);

        });

        frame.setLayout(new GridLayout(3, 1));

        frame.add(new JScrollPane(input));

        frame.add(generate);

        frame.add(new JScrollPane(output));

        frame.setVisible(true);

    }

    public static String generateTAC(String code) {

        // Simple demo TAC generation
```

```

return "t1 = c * d\n" +
    "t2 = b + t1\n" +
    "a = t2\n\n" +
    "Basic Block:\n" +
    "Block1 -> t1, t2, a\n\n" +
    "CFG:\n" +
    "[Start] -> Block1 -> [End]";
}
}

```

Output:


Lex + Yacc Front-End
 —
□
×

|

Evaluate

Result:



```
a = b + c * d;
```

Generate TAC

```
t1 = c * d  
t2 = b + t1  
a = t2
```

Basic Block:
Block1 -> t1, t2, a

CFG:
[Start] -> Block1 -> [End]

3. RESULT ANALYSIS

The system successfully converts input statements into intermediate representation and analyzes control flow.

Observations

- Expressions are converted into **Three Address Code (TAC)**
- Temporary variables (t1, t2) are generated correctly
- Basic blocks group sequential instructions
- CFG shows flow of execution clearly
- GUI allows easy interaction

Example Input

a = b + c * d;

Generated Output

t1 = c * d

t2 = b + t1

a = t2

Basic Block:

Block1 -> t1, t2, a

CFG:

[Start] -> Block1 -> [End]

Conclusion

This case study demonstrates how high-level code is transformed into intermediate representation and how program flow is analyzed using basic blocks and CFG. It forms a strong foundation for compiler optimization techniques.

4. REFERENCES

[1] Compilers: Principles, Techniques and Tools, A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson, 2006.

[2] Oracle Corporation, “Java Documentation,” [Online]. Available: <https://docs.oracle.com>. [Accessed: May 5, 2026].

[3] Osmania University, “Compiler Design Lecture Notes (OU Syllabus),” [Online].

[4] GeeksforGeeks, “Compiler Design Tutorials,” [Online]. Available: <https://www.geeksforgeeks.org/compiler-design-tutorials/>. [Accessed: May 5, 2026].

[5] Wikipedia, “Code Generation (Compiler),” [Online]. Available: [https://en.wikipedia.org/wiki/Code_generation_\(compiler\)](https://en.wikipedia.org/wiki/Code_generation_(compiler)). [Accessed: May 7, 2026].